T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010/ Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	September 17, 2026	**DATE SUBMITTED:**	September 21, 2026
NAME:	Regala Reyes Torres Santos	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 9.1 Hands-On Activity 9.1 Tree ADT

1. PROCEDURE OUTPUT

ILO A

Code + Console Screenshot

```cpp
#include <iostream>
#include "binaryGenTree.h"

int main() {
    // 1. Create Root Node 'A'
    BinaryGenTree<char>* tree = new BinaryGenTree<char>('A');

    // 2. Add Level 1 Children (B, C, D, E, F, G)
    BinaryGenTree<char>* nodeB = tree->insert('B');
    BinaryGenTree<char>* nodeC = tree->insert('C');
    BinaryGenTree<char>* nodeD = tree->insert('D');
    BinaryGenTree<char>* nodeE = tree->insert('E');
    BinaryGenTree<char>* nodeF = tree->insert('F');
    BinaryGenTree<char>* nodeG = tree->insert('G');

    // 3. Subtree D -> H
    nodeD->insert('H');

    // 4. Subtree E -> I, J (J -> P, Q)
    nodeE->insert('I');
    BinaryGenTree<char>* nodeJ = nodeE->insert('J');
    nodeJ->insert('P');
    nodeJ->insert('Q');

    // 5. Subtree F -> K, L, M
    nodeF->insert('K');
    nodeF->insert('L');
    nodeF->insert('M');

    // 6. Subtree G -> N, O
    nodeG->insert('N');
    nodeG->insert('O');

    // --- Visual Tree Structure ---
    std::cout << "Visual Tree Structure:\n";
    tree->printTree();
    std::cout << "\n";
```

```cpp
#ifndef BINARYGENTREE_H
#define BINARYGENTREE_H

#include <iostream>
#include <string>

template <typename T>
class BinaryGenTree {
private:
    T key;
    BinaryGenTree<T>* firCld;
    BinaryGenTree<T>* nxtSib;

    bool preOrder(T value);
    bool postOrder(T value);
    bool inOrder(T value);

public:
    // Constructor
    BinaryGenTree(T value);

    // Destructor
    ~BinaryGenTree();

    BinaryGenTree<T>* insert(T value);
    BinaryGenTree<T>* find(T value);


    void preOrder();
    void postOrder();
    void inOrder();

    void printTree(int depth = 0);
    void findData(int choice, T value);
};

template <typename T>
BinaryGenTree<T>::BinaryGenTree(T value) {
    key = value;
    firCld = nullptr;
    nxtSib = nullptr;
}

template <typename T>
BinaryGenTree<T>::~BinaryGenTree() {
    delete firCld;
    delete nxtSib;
}

template <typename T>
BinaryGenTree<T>* BinaryGenTree<T>::insert(T value) {
    BinaryGenTree<T>* node = new BinaryGenTree<T>(value);

    if (firCld == nullptr) {
        firCld = node;
    } else {
        BinaryGenTree<T>* current = firCld;
        while (current->nxtSib != nullptr) {
            current = current->nxtSib;
        }
        current->nxtSib = node;
    }
    return node;
}

template <typename T>
void BinaryGenTree<T>::printTree(int depth) {
    for (int i = 0; i < depth; i++) {
        std::cout << "    ";
    }
    std::cout << key << "\n";
    for (BinaryGenTree<T>* child = firCld; child != nullptr; child = child->nxtSib) {
        child->printTree(depth + 1);
    }
}

#endif
```

Observations	In this Code Implementation, we have implemented the general tree hierarchy is implemented using the First-Child / Next-Sibling pointer

	representation that is used in the BinaryGenTree class template declared in "binaryGenTree.h". Specifically, each node stores the data key, firCld that points to the main child, and nxtSib that links to the next node in the linked list of siblings. The tree was built dynamically in main() starting from the root node 'A' adding level-1 children (B, C, D, E, F, G) and the subtrees (D -> H; E -> I, J where J -> P, Q; F -> K, L, M; and G -> N, O) using the insert() method. The tree can be visualized using the printTree(int depth = 0) function which indents the four spaces per depth level, outputs the key, and runs the loop for (BinaryGenTree child = firCld; child != nullptr; child = child->nxtSib) recursively calling printTree(depth + 1) on the child nodes resulting in the header "Visual Tree Structure:" followed by the indented character tree.

Table 9.1 (Task 1)

Node	Height	Depth
A	0	3
B	1	0
C	1	0
D	1	1
E	1	2
F	1	1
G	1	1
H	2	0
I	2	0
J	2	1
K	2	0
L	2	0
M	2	0
N	2	0

P	**3**	**0**
Q	**3**	**0**

Table 9.2 (Task 2)

ILO B

Pre - Order	**A, B, C, H, D, E, I, J, P, F, Q, K, L, M, G, N**
In - Order	**B, C, H, D, Q, I, P, J, K, E, L, M, N, G, A**
Post - Order	**H, Q, P, J, I, M, L, K, G, N, F, E, D, C, B, A**

Table 9.3 (Task 3.1)

Code + Console Screenshot

```cpp
#include <iostream>
#include "binaryGenTree.h"

int main() {
    // 1. Create Root Node 'A'
    BinaryGenTree<char>* tree = new BinaryGenTree<char>('A');

    // 2. Add Level 1 Children (B, C, D, E, F, G)
    BinaryGenTree<char>* nodeB = tree->insert('B');
    BinaryGenTree<char>* nodeC = tree->insert('C');
    BinaryGenTree<char>* nodeD = tree->insert('D');
    BinaryGenTree<char>* nodeE = tree->insert('E');
    BinaryGenTree<char>* nodeF = tree->insert('F');
    BinaryGenTree<char>* nodeG = tree->insert('G');

    // 3. Subtree D -> H
    nodeD->insert('H');

    // 4. Subtree E -> I, J (J -> P, Q)
    nodeE->insert('I');
    BinaryGenTree<char>* nodeJ = nodeE->insert('J');
    nodeJ->insert('P');
    nodeJ->insert('Q');

    // 5. Subtree F -> K, L, M
    nodeF->insert('K');
    nodeF->insert('L');
    nodeF->insert('M');

    // 6. Subtree G -> N, O
    nodeG->insert('N');
    nodeG->insert('O');

    // --- Visual Tree Structure ---
    std::cout << "Visual Tree Structure:\n";
    tree->printTree();
    std::cout << "\n";
```

```h
#ifndef BINARYGENTREE_H
#define BINARYGENTREE_H

#include <iostream>
#include <string>

template <typename T>
class BinaryGenTree {
private:
    T key;
    BinaryGenTree<T>* firCld;
    BinaryGenTree<T>* nxtSib;

    bool preOrder(T value);
    bool postOrder(T value);
    bool inOrder(T value);

public:
    // Constructor
    BinaryGenTree(T value);

    // Destructor
    ~BinaryGenTree();

    BinaryGenTree<T>* insert(T value);
    BinaryGenTree<T>* find(T value);


    void preOrder();
    void postOrder();
    void inOrder();

    void printTree(int depth = 0);
    void findData(int choice, T value);
};

template <typename T>
BinaryGenTree<T>::BinaryGenTree(T value) {
    key = value;
    firCld = nullptr;
    nxtSib = nullptr;
}

template <typename T>
BinaryGenTree<T>::~BinaryGenTree() {
    delete firCld;
    delete nxtSib;
}

template <typename T>
BinaryGenTree<T>* BinaryGenTree<T>::insert(T value) {
    BinaryGenTree<T>* node = new BinaryGenTree<T>(value);

    if (firCld == nullptr) {
        firCld = node;
    } else {
        BinaryGenTree<T>* current = firCld;
        while (current->nxtSib != nullptr) {
            current = current->nxtSib;
        }
        current->nxtSib = node;
    }
    return node;
}
```

```cpp

// Pre-order
template <typename T>
void BinaryGenTree<T>::preOrder() {
    std::cout << key << " ";
    for (BinaryGenTree<T>* child = firCld; child != nullptr; child = child->nxtSib) {
        child->preOrder();
    }
}

// Post-order
template <typename T>
void BinaryGenTree<T>::postOrder() {
    for (BinaryGenTree<T>* child = firCld; child != nullptr; child = child->nxtSib) {
        child->postOrder();
    }
    std::cout << key << " ";
}

// In-order
template <typename T>
void BinaryGenTree<T>::inOrder() {
    if (firCld == nullptr) {
        std::cout << key << " ";
        return;
    }
    firCld->inOrder();
    std::cout << key << " ";
    for (BinaryGenTree<T>* child = firCld->nxtSib; child != nullptr; child = child->nxtSib) {
        child->inOrder();
    }
}

```

```cpp
template <typename T>
void BinaryGenTree<T>::printTree(int depth) {
    for (int i = 0; i < depth; i++) {
        std::cout << "    ";
    }
    std::cout << key << "\n";
    for (BinaryGenTree<T>* child = firCld; child != nullptr; child = child->nxtSib) {
        child->printTree(depth + 1);
    }
}

// Pre-order
template <typename T>
bool BinaryGenTree<T>::preOrder(T value) {
    if (key == value) {
        return true;
    }
    for (BinaryGenTree<T>* child = firCld; child != nullptr; child = child->nxtSib) {
        if (child->preOrder(value)) {
            return true;
        }
    }
    return false;
}

// Post-order
template <typename T>
bool BinaryGenTree<T>::postOrder(T value) {
    for (BinaryGenTree<T>* child = firCld; child != nullptr; child = child->nxtSib) {
        if (child->postOrder(value)) {
            return true;
        }
    }
    return key == value;
}
```

	
Observations	In this code and output that I've created below, I have implemented **tree traversal algorithms for a general tree with a left-child, right-sibling binary representation which is defined in the class template BinarySearchTree (located in the header file "binaryTreeTsk3_2.h" or "binaryTreeSuppl3_2.h", respectively)**. The program builds a **character-based tree at the dynamically allocated root node 'A' (by new BinarySearchTree('A')) attached to its descendant nodes ('B' through 'Q', respectively) using helper functions addChild() and addSibling()**. To explore the elements of the tree, **three generic traversal member functions (preOrder(), inOrder(), and postOrder())** have been implemented in the program, using **recursive operations over the left (child) and right (sibling) pointers**. These functions print each node's key separated by a space according to the traversals' visit sequences which are **root-first (key, left, right), binary in-order (left, key, right), and child/sibling-first (left, right, key)**. Calling **these functions in sequence (in main()) would display the headers "Pre-Order Traversal: ", "In-Order Traversal: ", and "Post-Order Traversal: " followed by the respective generated sequence outputs (A B C D H E I J P Q F K L M G N, B C H D I P Q J E K L M F N G A, and H Q P J I M L K N G F E D C B A)**. Comparing **these results of the program with the reference table would reveal a few misinputs which the reference table wrongly places the H before D in pre-order (H, D), shifts Q and K out of the sequence during in-order, and reverses the child-before-parent ordering of N and G in post-order (G, N instead of N, G)**.

Table 9.4 (Task 3.2)

Code + Console Screenshot

```cpp
#include <iostream>
#include "binaryTreeTsk3_3_4.h"

int main() {
    // Root Node A
    BinarySearchTree<char>* tree = new BinarySearchTree<char>('A');

    // Add Level 1 Children (B, C, D, E, F, G)
    BinarySearchTree<char>* nodeB = tree->addChild('B');
    BinarySearchTree<char>* nodeC = tree->addChild('C');
    BinarySearchTree<char>* nodeD = tree->addChild('D');
    BinarySearchTree<char>* nodeE = tree->addChild('E');
    BinarySearchTree<char>* nodeF = tree->addChild('F');
    BinarySearchTree<char>* nodeG = tree->addChild('G');

    // Subtree D -> H
    nodeD->addChild('H');

    // Subtree E -> I, J (J -> P, Q)
    nodeE->addChild('I');
    BinarySearchTree<char>* nodeJ = nodeE->addChild('J');
    nodeJ->addChild('P');
    nodeJ->addChild('Q');

    // Subtree F -> K, L, M
    nodeF->addChild('K');
    nodeF->addChild('L');
    nodeF->addChild('M');

    // Subtree G -> N
    nodeG->addChild('N');

    // Test findData searches
    tree->findData("PREORDER", 'J');  // Output: J was found!
    tree->findData("INORDER", 'P');   // Output: P was found!
    tree->findData("POSTORDER", 'Z'); // Output: (none)

    return 0;
}
```

```cpp
#ifndef BINARYTREETSK3_3_4_H
#define BINARYTREETSK3_3_4_H

#include <iostream>
#include <string>
#include <algorithm>

template <typename T>
class BinarySearchTree {
private:
    T key;
    // left = First Child, right = Next Sibling
    BinarySearchTree<T>* left;
    BinarySearchTree<T>* right;

    // Helper search routines for each traversal order
    bool searchPreOrder(T keyToFind);
    bool searchInOrder(T keyToFind);
    bool searchPostOrder(T keyToFind);

public:
    // Constructor
    BinarySearchTree(T value);

    // Tree-building helpers
    BinarySearchTree<T>* addChild(T value);
    BinarySearchTree<T>* addSibling(T value);

    // Visualization method
    void printTree(int depth = 0);

    // Traversing methods
    void preOrder();
    void inOrder();
    void postOrder();

    // Search method
    void findData(std::string choice, T keyToFind);
};

```

```cpp

template <typename T>
BinarySearchTree<T>::BinarySearchTree(T value) {
    key = value;
    left = nullptr;
    right = nullptr;
}

// Adds a child (first child goes to 'left')
template <typename T>
BinarySearchTree<T>* BinarySearchTree<T>::addChild(T value) {
    if (left == nullptr) {
        left = new BinarySearchTree<T>(value);
        return left;
    } else {
        return left->addSibling(value);
    }
}

// Adds a sibling (subsequent children go to 'right')
template <typename T>
BinarySearchTree<T>* BinarySearchTree<T>::addSibling(T value) {
    BinarySearchTree<T>* current = this;
    while (current->right != nullptr) {
        current = current->right;
    }
    current->right = new BinarySearchTree<T>(value);
    return current->right;
}

```

```cpp
// Prints visual tree structure
template <typename T>
void BinarySearchTree<T>::printTree(int depth) {
    std::string indent(depth * 4, ' ');

    if (depth > 0) {
        std::cout << indent << "|-- " << key << "\n";
    } else {
        std::cout << key << "\n";
    }

    if (left != nullptr) {
        BinarySearchTree<T>* currentChild = left;
        while (currentChild != nullptr) {
            currentChild->printTree(depth + 1);
            currentChild = currentChild->right;
        }
    }
}

template <typename T>
void BinarySearchTree<T>::preOrder() {
    std::cout << key << " ";
    if (left != nullptr) left->preOrder();
    if (right != nullptr) right->preOrder();
}

template <typename T>
void BinarySearchTree<T>::inOrder() {
    if (left != nullptr) left->inOrder();
    std::cout << key << " ";
    if (right != nullptr) right->inOrder();
}

```

```cpp
template <typename T>
void BinarySearchTree<T>::postOrder() {
    if (left != nullptr) left->postOrder();
    std::cout << key << " ";
    if (right != nullptr) right->postOrder();
}

// --- SEARCH HELPERS ---

// Pre-Order Search: Root -> Left -> Right
template <typename T>
bool BinarySearchTree<T>::searchPreOrder(T keyToFind) {
    if (key == keyToFind) return true;
    if (left != nullptr && left->searchPreOrder(keyToFind)) return true;
    if (right != nullptr && right->searchPreOrder(keyToFind)) return true;
    return false;
}

// In-Order Search: Left -> Root -> Right
template <typename T>
bool BinarySearchTree<T>::searchInOrder(T keyToFind) {
    if (left != nullptr && left->searchInOrder(keyToFind)) return true;
    if (key == keyToFind) return true;
    if (right != nullptr && right->searchInOrder(keyToFind)) return true;
    return false;
}

// Post-Order Search: Left -> Right -> Root
template <typename T>
bool BinarySearchTree<T>::searchPostOrder(T keyToFind) {
    if (left != nullptr && left->searchPostOrder(keyToFind)) return true;
    if (right != nullptr && right->searchPostOrder(keyToFind)) return true;
    if (key == keyToFind) return true;
    return false;
}

```

```cpp
// --- FIND DATA METHOD ---
template <typename T>
void BinarySearchTree<T>::findData(std::string choice, T keyToFind) {
    // Convert choice string to uppercase for case-insensitive comparison
    std::string mode = choice;
    std::transform(mode.begin(), mode.end(), mode.begin(), ::toupper);

    bool found = false;

    if (mode == "PREORDER" || mode == "PRE-ORDER" || mode == "PRE") {
        found = searchPreOrder(keyToFind);
    } else if (mode == "INORDER" || mode == "IN-ORDER" || mode == "IN") {
        found = searchInOrder(keyToFind);
    } else if (mode == "POSTORDER" || mode == "POST-ORDER" || mode == "POST") {
        found = searchPostOrder(keyToFind);
    } else {
        std::cout << "Invalid traversal choice.\n";
        return;
    }

    if (found) {
        std::cout << keyToFind << " was found!\n";
    }
}

#endif
```

```
J was found!
P was found!
```

Observations	In the code as well as the output below, I've implemented the **dynamic search routines for a binary tree** represented by **Left-Child/Right-Sibling binary structure in the class template BinarySearchTree in the header file "binaryTreeTsk3_3_4.h"**. I've built the **character hierarchy under the root node 'A' in the main() by the addChild() and addSibling() functions**. Also, I've enabled the **searching capabilities of this binary tree by three private recursive helper functions** which are the **searchPreOrder(), searchInOrder(), and searchPostOrder()**, that **traverse the binary tree's left (child) and right (sibling) pointers to check if the key is existed**. Moreover, the public member function **findData(std::string choice, T keyToFind) transforms the mode string to uppercase by using std::transform to perform case-insensitive comparison before calling the appropriate traversal helper function**. **If the key was found, then the findData() function prints "[key] was found!",**

	or else, no printing will happen when it fails to find the letter. The calls to **tree->findData("PREORDER", 'J') and tree->findData("INORDER", 'P') in the main() successfully find the targets** and **print out "J was found!" and "P was found!" to the console**, while the call to **tree->findData("POSTORDER", 'Z') correctly prints nothing since 'Z' is not present in the tree structure itself**.

Table 9.5 (Task 3.3)

Code + Console Screenshot	

```cpp
#include <iostream>
#include "binaryTreeTsk3_3_4.h"

int main() {
    // Root Node A
    BinarySearchTree<char>* tree = new BinarySearchTree<char>('A');

    // Add Level 1 Children (B, C, D, E, F, G)
    BinarySearchTree<char>* nodeB = tree->addChild('B');
    BinarySearchTree<char>* nodeC = tree->addChild('C');
    BinarySearchTree<char>* nodeD = tree->addChild('D');
    BinarySearchTree<char>* nodeE = tree->addChild('E');
    BinarySearchTree<char>* nodeF = tree->addChild('F');
    BinarySearchTree<char>* nodeG = tree->addChild('G');

    // Subtree D -> H
    nodeD->addChild('H');

    // Subtree E -> I, J (J -> P, Q)
    nodeE->addChild('I');
    BinarySearchTree<char>* nodeJ = nodeE->addChild('J');
    nodeJ->addChild('P');
    nodeJ->addChild('Q');

    // Subtree F -> K, L, M
    nodeF->addChild('K');
    nodeF->addChild('L');
    nodeF->addChild('M');

    // Subtree G -> N
    nodeG->addChild('N');

    // Subtree G -> O
    nodeG->addChild('O');


    // Test findData searches
    tree->findData("PREORDER", 'J');  // Output: J was found!
    tree->findData("INORDER", 'P');   // Output: P was found!
    tree->findData("POSTORDER", 'O'); // Output: O was found!
    tree->findData("POSTORDER", 'Z'); // Output: (none)

    return 0;
}
```

```h
#ifndef BINARYTREETSK3_3_4_H
#define BINARYTREETSK3_3_4_H

#include <iostream>
#include <string>
#include <algorithm>

template <typename T>
class BinarySearchTree {
private:
    T key;
    // left = First Child, right = Next Sibling
    BinarySearchTree<T>* left;
    BinarySearchTree<T>* right;

    // Helper search routines for each traversal order
    bool searchPreOrder(T keyToFind);
    bool searchInOrder(T keyToFind);
    bool searchPostOrder(T keyToFind);

public:
    // Constructor
    BinarySearchTree(T value);

    // Tree-building helpers
    BinarySearchTree<T>* addChild(T value);
    BinarySearchTree<T>* addSibling(T value);

    // Visualization method
    void printTree(int depth = 0);

    // Traversing methods
    void preOrder();
    void inOrder();
    void postOrder();

    // Search method
    void findData(std::string choice, T keyToFind);
};

```

```h

template <typename T>
BinarySearchTree<T>::BinarySearchTree(T value) {
    key = value;
    left = nullptr;
    right = nullptr;
}

// Adds a child (first child goes to 'left')
template <typename T>
BinarySearchTree<T>* BinarySearchTree<T>::addChild(T value) {
    if (left == nullptr) {
        left = new BinarySearchTree<T>(value);
        return left;
    } else {
        return left->addSibling(value);
    }
}

// Adds a sibling (subsequent children go to 'right')
template <typename T>
BinarySearchTree<T>* BinarySearchTree<T>::addSibling(T value) {
    BinarySearchTree<T>* current = this;
    while (current->right != nullptr) {
        current = current->right;
    }
    current->right = new BinarySearchTree<T>(value);
    return current->right;
}

```

```cpp
// Prints visual tree structure
template <typename T>
void BinarySearchTree<T>::printTree(int depth) {
    std::string indent(depth * 4, ' ');

    if (depth > 0) {
        std::cout << indent << "|-- " << key << "\n";
    } else {
        std::cout << key << "\n";
    }

    if (left != nullptr) {
        BinarySearchTree<T>* currentChild = left;
        while (currentChild != nullptr) {
            currentChild->printTree(depth + 1);
            currentChild = currentChild->right;
        }
    }
}

template <typename T>
void BinarySearchTree<T>::preOrder() {
    std::cout << key << " ";
    if (left != nullptr) left->preOrder();
    if (right != nullptr) right->preOrder();
}

template <typename T>
void BinarySearchTree<T>::inOrder() {
    if (left != nullptr) left->inOrder();
    std::cout << key << " ";
    if (right != nullptr) right->inOrder();
}

template <typename T>
void BinarySearchTree<T>::postOrder() {
    if (left != nullptr) left->postOrder();
    std::cout << key << " ";
    if (right != nullptr) right->postOrder();
}

// --- SEARCH HELPERS ---

// Pre-Order Search: Root -> Left -> Right
template <typename T>
bool BinarySearchTree<T>::searchPreOrder(T keyToFind) {
    if (key == keyToFind) return true;
    if (left != nullptr && left->searchPreOrder(keyToFind)) return true;
    if (right != nullptr && right->searchPreOrder(keyToFind)) return true;
    return false;
}

// In-Order Search: Left -> Root -> Right
template <typename T>
bool BinarySearchTree<T>::searchInOrder(T keyToFind) {
    if (left != nullptr && left->searchInOrder(keyToFind)) return true;
    if (key == keyToFind) return true;
    if (right != nullptr && right->searchInOrder(keyToFind)) return true;
    return false;
}

// Post-Order Search: Left -> Right -> Root
template <typename T>
bool BinarySearchTree<T>::searchPostOrder(T keyToFind) {
    if (left != nullptr && left->searchPostOrder(keyToFind)) return true;
    if (right != nullptr && right->searchPostOrder(keyToFind)) return true;
    if (key == keyToFind) return true;
    return false;
}

// --- FIND DATA METHOD ---
template <typename T>
void BinarySearchTree<T>::findData(std::string choice, T keyToFind) {
    // Convert choice string to uppercase for case-insensitive comparison
    std::string mode = choice;
    std::transform(mode.begin(), mode.end(), mode.begin(), ::toupper);

    bool found = false;

    if (mode == "PREORDER" || mode == "PRE-ORDER" || mode == "PRE") {
        found = searchPreOrder(keyToFind);
    } else if (mode == "INORDER" || mode == "IN-ORDER" || mode == "IN") {
        found = searchInOrder(keyToFind);
    } else if (mode == "POSTORDER" || mode == "POST-ORDER" || mode == "POST") {
        found = searchPostOrder(keyToFind);
    } else {
        std::cout << "Invalid traversal choice.\n";
        return;
    }

    if (found) {
        std::cout << keyToFind << " was found!\n";
    }
}

#endif
```

```
J was found!
P was found!
O was found!
```

Observations	In the code as well as the output below is the same as the Task 3.3, I've implemented the **dynamic search routines for a binary tree** represented by **Left-Child/Right-Sibling binary structure in the class template BinarySearchTree in the header file "binaryTreeTsk3_3_4.h"**. I've built the **character hierarchy under the root node 'A' in the main() by the addChild() and addSibling() functions which the difference now from Task 3.3 is that I've added 'O' on the Subtree 'G'** by **nodeG->addChild('O')**. Now, I've enabled the **searching capabilities of this binary tree by three private recursive helper functions** which are the **searchPreOrder(), searchInOrder(), and searchPostOrder()**, that **traverse the binary tree's left (child) and right (sibling) pointers to check if the key is existed**. Moreover, the public member function **findData(std::string choice, T keyToFind) transforms the mode string to uppercase by using std::transform to perform case-insensitive comparison before calling the appropriate traversal helper function**. **If the key was found, then the findData() function prints "[key] was found!", or else, no printing will happen when it fails to find the letter**. The calls to **tree->findData("PREORDER", 'J') , tree->findData("INORDER", 'P'), and tree->findData("POSTORDER", 'O') in the main() successfully find the targets** and **print out "J was found!", "P was found!", and "O was found" to the console**, while the call to **tree->findData("POSTORDER", 'Z') correctly prints nothing since 'Z' is not present in the tree structure itself**.

Table 9.6 (Task 3.4)

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve given problems using the tree data structure's implementation in C++

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

```
4   int main() {
5       // values =  2, 3, 9, 18, 0, 1, 4, 5
6       BinarySearchTree<int>* tree = new BinarySearchTree<int>(2);
7       tree->insert(3);
8       tree->insert(9);
9       tree->insert(18);
10      tree->insert(0);
11      tree->insert(1);
12      tree->insert(4);
13      tree->insert(5);
```

Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.

```
15      // Tree structure:
16      /*          2
17                /   \
18              0      3
19               \      \
20                1      9
21                      /  \
22                     4    18
23                      \
24                       5
25      */
```

```
Pre-order:  2 0 1 3 9 4 5 18
Post-order: 1 0 5 4 18 9 3 2
In-order:   0 1 2 3 4 5 9 18
```

Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

```
106   // In-order
107   template <typename T>
108   void BinarySearchTree<T>::inOrder() {
109       if (left != nullptr) {
110           left->inOrder();
111       }
112       std::cout << key << " ";
113       if (right != nullptr) {
114           right->inOrder();
115       }
116   }
117
```

```
Post-order: 1 0 5 4 18 9 3 2
In-order:   0 1 2 3 4 5 9 18
```

Given the same input values above, what is the output with different traversal methods? For each output below, indicate your observation: is the output different from the pre-order and post-order traversal that you indicated in the diagrams shown in item #2.

Pre-order Traversal

```
132   // Pre-order
133   template <typename T>
134   bool BinarySearchTree<T>::preOrder(T value) {
135       if (key == value) {
136           return true;
137       }
138       if (left != nullptr && left->preOrder(value)) {
139           return true;
140       }
141       if (right != nullptr && right->preOrder(value)) {
142           return true;
143       }
144       return false;
145   }
```

```
Pre-order:  2 0 1 3 9 4 5 18
```

Post-order Traversal

```
147  // Post-order
148  template <typename T>
149  bool BinarySearchTree<T>::postOrder(T value) {
150      if (left != nullptr && left->postOrder(value)) {
151          return true;
152      }
153      if (right != nullptr && right->postOrder(value)) {
154          return true;
155      }
156      return key == value;
157  }
```

```
Post-order: 1 0 5 4 18 9 3 2
```

Observation	The console output for pre-order (2 0 1 3 9 4 5 18) and post-order (1 0 5 4 18 9 3 2) matched the sequences we traced from the diagram exactly. In-order was performed by the inOrder() function and the BST produced the fully sorted sequence 0 1 2 3 4 5 9 18.

CONCLUSION/LEARNINGS/ ANALYSIS	**Through this hands-on activity, I concluded that choosing between implementing Shell Sort, Merge Sort, and Quick Sort algorithms is dependent on the size of the data set, memory size allocation, and the design of the algorithm itself. I discovered that Shell Sort arranges data in a specific manner by initially setting a gap of size / 2 then dividing it by 2 (gap = gap / 2) and using insertion sort algorithm on gap intervals to minimize the distance between any two unsorted elements until a non-decreasing order is reached. Moreover, I found out that Merge Sort utilized recursion to break down an array into sub-arrays by setting a middle = (left + right) / 2 and merged them using a temporary array helper function merge(), which guarantees O(N logN) time complexity since merging split sub-arrays takes O(N) time and space complexity while dividing the array by halves (logN). In contrast, Quick Sort achieves O(N log N) time complexity using a "Lomuto partitioning scheme" that sets the last element as a pivot (arr[high]), swaps it to the correct position in the array (index pi = i + 1), and recursively sorts the sub-lists on either side of the pivot. By analyzing the performance characteristics of**

	these algorithms, I discovered that Quick Sort is faster in smaller data sets because the in-place simple partition looping that it employs does not require the additional memory-management overhead required by the Merge Sort Algorithm while at the same time being flexible enough to implement hybrid implementations that sort partitioned sub-lists using Shell Sort in the Problem 1. In my improvement that needs to be done, I want to improve my argument creation skills because it is needed for any sorting algorithms because there are times that I can't debug the code itself or the output didn't go as it should be so I need to improve this skill to apply this to the future lectures coding procedures efficiently.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **NONE**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **NONE**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **NONE**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.